Lecture 3 — Matching and Linear Programming

> World Finals Level · Prerequisites: *week-32* (max-flow, min-cut, bipartite matching), *nationals-05* (min-cost flow, Hopcroft–Karp, lower-bound modelling), *nationals-07* (Gaussian elimination) · Estimated time: 2 × 90 minutes

1. Overview

Flows solve a huge class of problems, but WF sets regularly contain problems just outside that class.

- **Weighted bipartite matching (assignment).** Min-cost flow solves it, but the Hungarian algorithm is shorter, faster ($O(n^2m)$ with a tiny constant) and gives you the dual potentials for free.
- **Matching in general (non-bipartite) graphs.** Augmenting-path search fails on odd cycles. Edmonds' blossom algorithm fixes this in $O(V^3)$.
- **Linear programming.** When variables are continuous and constraints linear, a simplex solver answers directly. More importantly, **LP duality** explains why max-flow equals min-cut, why Hungarian potentials work, and how to turn an awkward problem into a shortest-path or flow problem.
- **Flows with lower bounds (demands).** "Each edge must carry at least l units" reduces to a single max-flow saturation check.

All four implementations in Section 7 were tested against brute force: Hungarian against enumeration of all injections, blossom against a bitmask DP, simplex against vertex enumeration and against Hungarian on assignment LPs, and bounded flows against enumeration of all integral flows.

2. The Hungarian algorithm

2.1 Problem and LP view

We are given an $n \times m$ cost matrix `a` with $n \le m$. We must assign each row a distinct column and minimise the total cost. As an LP:

```
min Σ a_ij x_ij   s.t.   Σ_j x_ij = 1 ∀i,   Σ_i x_ij ≤ 1 ∀j,   x ≥ 0.
```

The constraint matrix is the incidence matrix of a bipartite graph, which is **totally unimodular**, so the LP has an integral optimal vertex. The dual has a variable `u_i` per row and `v_j` per column:

```
max Σ u_i + Σ v_j   s.t.   u_i + v_j ≤ a_ij.
```

(When n < m, the column constraints are inequalities, so the column duals must satisfy v_j ≤ 0 and unmatched columns need v_j = 0. The algorithm below starts from v = 0, only ever decreases v_j, and only for columns that are or become matched, so both conditions hold.) The **reduced cost** of cell (i, j) is `a_ij − u_i − v_j ≥ 0`. **Complementary slackness** says a primal/dual pair is optimal exactly when every used cell is **tight** (reduced cost 0). So the Hungarian algorithm's goal is to find feasible potentials together with a perfect matching of rows that uses only tight cells.

2.2 The algorithm

Rows are added one at a time. When adding row i, the algorithm grows an alternating tree from i:

- `minv[j]` is the smallest reduced cost of an edge from a tree row into column j, and `way[j]` remembers which tree column led to it. This is exactly Dijkstra's tentative distance with reduced costs as edge weights.
- Let δ be the smallest `minv[j]` over columns not yet in the tree. Add δ to `u` of every tree row and subtract δ from `v` of every tree column. Reduced costs inside the tree are unchanged. Reduced costs from tree rows to non-tree columns drop by δ, and stay ≥ 0 by the choice of δ. At least one becomes 0: a new tight edge.
- The column `j1` that achieved δ joins the tree. If it is free, we augment along the `way` pointers. Otherwise its matched row joins the tree and we continue.

Each step costs O(m) and adds a column to the tree, so adding one row costs O(nm) at most (really O(m · tree size)). The total is **$O(n^2m)$**, $O(n^3)$ for square matrices, with a very small constant: n = 1000 runs in well under a second.

2.3 Reading the answer

Column 0 is a virtual column used as the root of every search, and it is always in the tree. In each step, the k + 1 tree rows get +δ and the k real tree columns plus column 0 get −δ. So the dual objective over real rows and columns increases by δ and `v[0]` decreases by δ. At the end the dual is feasible and complementary slackness holds, so the optimal cost equals the dual objective, which equals **−v[0]**.

2.4 Variants

- **Maximisation**: negate the costs.
- **n > m**: transpose the matrix.
- **Forbidden cells**: use a large finite cost `BIG`, chosen so that `n · BIG` does not overflow, and check afterwards that no forbidden cell was used.
- **Potentials as a certificate**: the final `u`, `v` prove optimality. Some problems ask for the dual itself (for example, "assign prices to rows and columns such that …").

3. General matching: Edmonds' blossom algorithm

3.1 What breaks

Berge's theorem holds in every graph: a matching is maximum if and only if there is no augmenting path. The bipartite search labels vertices *even* (reached by an even-length alternating path from a free root) or *odd*, and only extends the search from even vertices. In a non-bipartite graph an edge may join **two even vertices**. This closes an odd alternating cycle, a *blossom*. The vertices on the cycle can be reached with *either* parity by going around the cycle in the appropriate direction, and the bipartite search misses augmenting paths that need the other direction. The slides contain a five-vertex example where the bipartite search fails.

3.2 Contraction

Edmonds' lemma. Let B be a blossom found during the search. Contract B into a single vertex. Then G has an augmenting path with respect to M if and only if the contracted graph G/B has one with respect to M/B. Moreover, a path in G/B lifts to a path in G by going around the blossom the right way.

The implementation never contracts physically. It keeps `base[v]`, the base vertex of the outermost blossom containing v, and treats all vertices with the same base as one vertex. When an edge (v, to) joins two even vertices:

1. compute `cur = lca(v, to)` in the alternating forest (using bases);
2. walk from v and from `to` up to `cur` (`markPath`), marking the blossoms passed and **re-pointing `par`** so that odd vertices on the cycle get a parent that lets the path go around the cycle;
3. set `base[i] = cur` for every vertex in a marked blossom, and push any vertex that was not yet even into the queue. All blossom vertices are now even.

When the search reaches a free vertex `to` from an even vertex, we augment. We walk back using `par` and `match`, flipping matched and unmatched edges.

3.3 Complexity and practicalities

There is one search per vertex. Each search processes O(V) blossoms, each relabelled in O(V), plus O(E) edge scans, for **O(V³)** total. A greedy initial matching usually leaves only a few free vertices, so in practice the algorithm is much faster than the bound. V = 500 on a dense graph takes milliseconds.

Alternatives worth knowing:

- **Tutte matrix (randomised).** Build a skew-symmetric matrix with `T[i][j] = x_ij`, `T[j][i] = −x_ij` for each edge, with random `x_ij` modulo a large prime. Then rank(T) = 2ν(G) with high probability. This takes ten lines on top of Gaussian elimination (*nationals-07*), and recovering the matching itself is possible but more involved.
- **Weighted general matching.** Edmonds' algorithm with dual variables on blossoms, O(V³). The code is 200+ lines; keep a tested copy in the notebook (Library Checker has a judge for it) rather than writing it in a contest.

4. Linear programming and duality

4.1 Standard form

Our solver takes:

```
maximise cᵀx   subject to   A x ≤ b,   x ≥ 0.
```

Converting anything else is mechanical: an equality becomes two inequalities, a free variable becomes $x^+ - x^-$, a minimisation becomes maximising $-c^\mathsf{T}x$, and a $\geq$ constraint is multiplied by -1 (which may make b negative, handled by phase 1). If the feasible region is non-empty and the objective is bounded, an optimum is attained at a **vertex** of the polytope.

4.2 Duality

Primal	Dual
max $c^\mathsf{T}x$	min $b^\mathsf{T}y$
$Ax \leq b$	$A^\mathsf{T}y \geq c$
$x \geq 0$	$y \geq 0$

- **Weak duality.** If x and y are feasible, $c^\mathsf{T}x \leq y^\mathsf{T}Ax \leq b^\mathsf{T}y$. Every dual solution bounds the primal.
- **Strong duality.** If the primal has a finite optimum, so does the dual, and they are equal.
- **Complementary slackness.** Feasible x and y are both optimal if and only if $y_i > 0 \Rightarrow (Ax)_i = b_i$ and $x_j > 0 \Rightarrow (A^\mathsf{T}y)_j = c_j$.

Recipe for taking a dual: one dual variable per primal constraint, one dual constraint per primal variable. Transpose A and swap b and c. The sign of each dual variable follows the sense of its primal constraint (≤ gives y ≥ 0, = gives a free y), and the sense of each dual constraint follows the sign of its primal variable (x ≥ 0 gives ≥, free x gives =).

4.3 Duality in contest problems

You have already used LP duality without naming it:

- **Max-flow / min-cut.** The dual of the max-flow LP is a fractional cut LP whose optimal vertices are integral cuts.
- **König's theorem.** In bipartite graphs, maximum matching = minimum vertex cover.
- **Shortest paths.** `max d_t − d_s` subject to `d_v − d_u ≤ w_uv` is the dual of the min-cost s–t path. Any system of **difference constraints** is a shortest-path problem: negative cycle means infeasible.
- **Min-cost flow** potentials (Johnson reweighting) are dual variables, and the Hungarian `u, v` are the same thing.

The practical lesson is: **when the primal is awkward, write the dual.** A common WF pattern is "assign numbers to vertices so that each edge satisfies an inequality, and optimise a linear function". After dualising, that often becomes min-cost flow. This is the standard "dual of min-cost flow" technique for objectives over constraints of the form `x_v − x_u ≤ w`.

4.4 The simplex method

Add slack variables so that `A x + s = b`. A **basis** is a set of m variables whose columns are linearly independent. Setting the other (non-basic) variables to 0 gives a vertex. When b ≥ 0, the slack basis (`x = 0`) is feasible.

A **pivot** swaps one non-basic variable into the basis:

- **Entering column.** Any non-basic variable with negative reduced cost in the objective row: increasing it improves the objective. Our code takes the most negative one (Dantzig's rule), breaking ties by variable index.
- **Leaving row.** The *minimum ratio test*: among rows with a positive entry in the entering column, choose the one minimising `b_i / D[i][s]`. This is the first constraint to become tight. If no row qualifies, the LP is **unbounded**.
- Then do a Gauss–Jordan elimination step on the tableau.

On **degenerate** LPs (several constraints tight at the same vertex), pivots can make zero progress and in principle **cycle**. Bland's rule (choose the smallest eligible index for both entering and leaving) provably prevents cycling. Dantzig's rule with index tie-breaking, as in the code, is faster in practice and cycles only on contrived inputs. If you ever see a time limit exceeded on a degenerate LP, switch to Bland.

Phase 1. If some `b_i < 0`, add an artificial variable x_0 with coefficient -1 in every row. Pivoting x_0 into the row with the most negative b makes all right-hand sides non-negative. Then minimise x_0. If its optimum is positive, the LP is **infeasible**. Otherwise, pivot x_0 out of the basis if it is still basic (at value 0), and continue with the real objective (phase 2).

Complexity. The worst case is exponential (Klee–Minty), but in practice the number of pivots is a small multiple of m, each costing O(mn). With doubles, LPs with a few hundred constraints and variables are fine.

Precision. Use `EPS ≈ 1e–9`, keep input magnitudes in a moderate range (rescale if necessary), and remember that the result is only integral if the constraint matrix is totally unimodular (flows, bipartite matching, interval scheduling matrices). Codeforces 375E *Red and Black Tree* is a well-known problem commonly solved by running simplex on its LP formulation, whose optimum turns out to be integral.

5. Flows with lower bounds

Each edge `u → v` must carry flow in `[l, r]`.

5.1 Feasible circulation

1. Replace the edge by one with capacity `r − l`.
2. Pretend that `l` units already flow along it. Then v has `l` units of excess and u has a deficit of `l`. Accumulate `excess[v] += l`, `excess[u] −= l`.
3. Add a super source S′ with an edge `S′ → v` of capacity `excess[v]` for every v with positive excess, and an edge `v → T′` of capacity `−excess[v]` for every v with negative excess.
4. A feasible circulation exists **if and only if** max-flow(S′, T′) saturates every S′ edge.

Proof sketch. A feasible circulation f gives a flow `f − l` on the reduced edges. At each vertex, `f − l` has a net imbalance of exactly `−excess[v]`, which the S′/T′ edges absorb, saturating them. Conversely, a saturating S′T′ flow g gives `l + g`, which satisfies conservation and the bounds.

5.2 With a source and a sink

- **Feasible s–t flow.** Add an edge `t → s` with infinite capacity and look for a feasible circulation.
- **Maximum flow with bounds.** After the feasibility check, the flow on `t → s` is a feasible value. Delete that edge and add `max-flow(s, t)` computed in the residual graph. The S′ and T′ edges are saturated and cannot be used.
- **Minimum flow with bounds.** After feasibility, delete `t → s` and subtract `max-flow(t, s)` in the residual graph. That pushes flow back from t to s as much as the lower bounds allow.
- **Flow on edge i** is `l_i` plus the flow on its reduced edge.

5.3 Modelling patterns

- **Matrix rounding.** Rows and columns with sum constraints in `[a, b]` become a bipartite network with bounded edges.
- **Staffing and scheduling with minimum coverage.** Lower bounds on time-interval edges.
- **Degree-constrained subgraphs** (each vertex has degree in `[lo, hi]`), in bipartite graphs.
- **Codeforces 704D *Captain America*.** Each point is coloured red or blue at different costs, and each row and column has a bound on |#red − #blue|. This is a bounded flow over rows and columns.

6. How these show up at World Finals

- Assignment-type subproblems hidden in geometry or scheduling (match n robots to n targets minimising total distance) are solved by Hungarian, or by Hungarian inside a binary search for bottleneck variants.
- WF 2015 *Catering* is min-cost flow where every request must be served, which you can also express with lower bounds of 1 on request nodes.
- General-graph matching appears when "pairs" are formed from a single set: pairing people, tiles, or endpoints.
- LPs appear rarely as explicit LPs, but duality-based reformulations (difference constraints, potentials) appear regularly.

7. Reference implementations

7.1 Hungarian algorithm (CSES 2129 *Task Assignment* format)

Input: n, then the n × n cost matrix. Output: the minimum cost, then the n assigned pairs.

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// a is (n+1) x (m+1), 1-indexed, n <= m. Returns {cost, match} with row i -> column
match[i].
pair<ll, vector<int>> hungarian(const vector<vector<ll>>& a) {
    int n = a.size() - 1, m = a[0].size() - 1;
    const ll INF = numeric_limits<ll>::max() / 4;
    vector<ll> u(n + 1), v(m + 1);             // potentials: u[i] + v[j] <= a[i][j]
    vector<int> p(m + 1), way(m + 1);           // p[j] = row matched to column j (0 =
none)
    for (int i = 1; i <= n; i++) {
        p[0] = i;                               // column 0 is a virtual start
```

```
        int j0 = 0;
        vector<ll> minv(m + 1, INF);            // minv[j] = min reduced cost into column
j
        vector<char> used(m + 1, false);
        do {                                    // Dijkstra on reduced costs
            used[j0] = true;
            int i0 = p[j0], j1 = 0;
            ll delta = INF;
            for (int j = 1; j <= m; j++) if (!used[j]) {
                ll cur = a[i0][j] - u[i0] - v[j];
                if (cur < minv[j]) minv[j] = cur, way[j] = j0;
                if (minv[j] < delta) delta = minv[j], j1 = j;
            }
            for (int j = 0; j <= m; j++)        // shift potentials by delta
                if (used[j]) u[p[j]] += delta, v[j] -= delta;
                else minv[j] -= delta;
            j0 = j1;
        } while (p[j0] != 0);                   // until we reach a free column
        do {                                    // augment along the way[] pointers
            int j1 = way[j0];
            p[j0] = p[j1];
            j0 = j1;
        } while (j0);
    }
    vector<int> match(n + 1);
    for (int j = 1; j <= m; j++) if (p[j]) match[p[j]] = j;
    return {-v[0], match};
}

int main() {
    int n;
    cin >> n;
    vector<vector<ll>> a(n + 1, vector<ll>(n + 1));
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++) cin >> a[i][j];
    auto [cost, match] = hungarian(a);
    cout << cost << "\n";
    for (int i = 1; i <= n; i++) cout << i << " " << match[i] << "\n";
}
```

7.2 Edmonds' blossom algorithm (Library Checker *Matching on General Graph* format)

Input: `N M`, then M edges `u v` (0-indexed). Output: the matching size and the matched pairs.

```
#include <bits/stdc++.h>
using namespace std;

struct Blossom {
    int n;
    vector<vector<int>> g;
    vector<int> match, par, base, q;
```

```
    vector<char> used, inBlossom;
    Blossom(int n) : n(n), g(n), match(n, -1), par(n), base(n), used(n), inBlossom(n) {}
    void addEdge(int a, int b) { g[a].push_back(b); g[b].push_back(a); }

    int lca(int a, int b) {                    // LCA of two even vertices in the
alternating forest
        vector<char> seen(n, false);
        for (;;) {
            a = base[a]; seen[a] = true;
            if (match[a] == -1) break;         // reached the root
            a = par[match[a]];
        }
        for (;;) {
            b = base[b];
            if (seen[b]) return b;
            b = par[match[b]];
        }
    }
    void markPath(int v, int b, int child) {   // walk v up to blossom base b
        while (base[v] != b) {
            inBlossom[base[v]] = inBlossom[base[match[v]]] = true;
            par[v] = child;
            child = match[v];
            v = par[match[v]];
        }
    }
    int findPath(int root) {                   // BFS for an augmenting path from root
        fill(used.begin(), used.end(), false);
        fill(par.begin(), par.end(), -1);
        iota(base.begin(), base.end(), 0);
        used[root] = true;
        q.assign(1, root);
        for (size_t qh = 0; qh < q.size(); qh++) {
            int v = q[qh];
            for (int to : g[v]) {
                if (base[v] == base[to] || match[v] == to) continue;
                if (to == root || (match[to] != -1 && par[match[to]] != -1)) {
                    int cur = lca(v, to);      // odd cycle: contract the blossom
                    fill(inBlossom.begin(), inBlossom.end(), false);
                    markPath(v, cur, to);
                    markPath(to, cur, v);
                    for (int i = 0; i < n; i++)
                        if (inBlossom[base[i]]) {
                            base[i] = cur;
                            if (!used[i]) used[i] = true, q.push_back(i);
                        }
                } else if (par[to] == -1) {    // grow the tree
                    par[to] = v;
                    if (match[to] == -1) return to;   // augmenting path found
                    used[match[to]] = true;
                    q.push_back(match[to]);
                }
            }
        }
        return -1;
```

```
    }
    int solve() {
        int res = 0;
        for (int v = 0; v < n; v++)             // greedy warm start (big constant-factor
win)
            if (match[v] == -1)
                for (int to : g[v]) if (match[to] == -1) { match[to] = v; match[v] = to;
res++; break; }
        for (int v = 0; v < n; v++) if (match[v] == -1) {
            int x = findPath(v);
            if (x != -1) res++;
            while (x != -1) {                   // flip the augmenting path
                int pv = par[x], ppv = match[pv];
                match[x] = pv; match[pv] = x;
                x = ppv;
            }
        }
        return res;
    }
};

int main() {
    int n, m;
    cin >> n >> m;
    Blossom b(n);
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        if (u != v) b.addEdge(u, v);
    }
    cout << b.solve() << "\n";
    for (int v = 0; v < n; v++)
        if (b.match[v] > v) cout << v << " " << b.match[v] << "\n";
}
```

7.3 Two-phase simplex

Input: `n m`, the objective `c` (n numbers), then m rows each containing n coefficients followed by `b_i`. Output: `INFEASIBLE`, `UNBOUNDED`, or the optimum and an optimal x.

```
#include <bits/stdc++.h>
using namespace std;

// maximize c^T x subject to A x <= b, x >= 0.
// Returns -inf if infeasible, +inf if unbounded. Dantzig pivoting with index tie-breaks.
struct LPSolver {
    using T = double;
    static constexpr T EPS = 1e-9;
    int m, n;
    vector<int> N, B;                           // non-basic / basic variable ids
    vector<vector<T>> D;                        // tableau, (m+2) x (n+2)
    LPSolver(const vector<vector<T>>& A, const vector<T>& b, const vector<T>& c)
```

```
        : m(b.size()), n(c.size()), N(n + 1), B(m), D(m + 2, vector<T>(n + 2)) {
        for (int i = 0; i < m; i++) for (int j = 0; j < n; j++) D[i][j] = A[i][j];
        for (int i = 0; i < m; i++) { B[i] = n + i; D[i][n] = -1; D[i][n + 1] = b[i]; }
        for (int j = 0; j < n; j++) { N[j] = j; D[m][j] = -c[j]; }
        N[n] = -1; D[m + 1][n] = 1;             // column n = artificial variable for
phase 1
    }
    void pivot(int r, int s) {
        T inv = 1 / D[r][s];
        for (int i = 0; i < m + 2; i++) if (i != r && fabs(D[i][s]) > EPS) {
            T f = D[i][s] * inv;
            for (int j = 0; j < n + 2; j++) D[i][j] -= D[r][j] * f;
            D[i][s] = D[r][s] * f;
        }
        for (int j = 0; j < n + 2; j++) if (j != s) D[r][j] *= inv;
        for (int i = 0; i < m + 2; i++) if (i != r) D[i][s] *= -inv;
        D[r][s] = inv;
        swap(B[r], N[s]);
    }
    bool simplex(int phase) {
        int x = m + phase - 1;                  // objective row for this phase
        for (;;) {
            int s = -1;                         // entering: most negative reduced cost,
ties by id
            for (int j = 0; j <= n; j++) {
                if (N[j] == -phase) continue;
                if (s == -1 || make_pair(D[x][j], N[j]) < make_pair(D[x][s], N[s])) s =
j;
            }
            if (D[x][s] >= -EPS) return true;   // optimal
            int r = -1;                         // leaving: min ratio test, ties by id
            for (int i = 0; i < m; i++) {
                if (D[i][s] <= EPS) continue;
                if (r == -1 || make_pair(D[i][n + 1] / D[i][s], B[i]) <
                               make_pair(D[r][n + 1] / D[r][s], B[r])) r = i;
            }
            if (r == -1) return false;          // unbounded
            pivot(r, s);
        }
    }
    T solve(vector<T>& x) {
        int r = 0;
        for (int i = 1; i < m; i++) if (D[i][n + 1] < D[r][n + 1]) r = i;
        if (m > 0 && D[r][n + 1] < -EPS) {      // phase 1: origin is infeasible
            pivot(r, n);
            if (!simplex(2) || D[m + 1][n + 1] < -EPS) return -
numeric_limits<T>::infinity();
            for (int i = 0; i < m; i++) if (B[i] == -1) {   // drive artificial out of
basis
                int s = 0;
                for (int j = 1; j <= n; j++)
                    if (s == -1 || make_pair(D[i][j], N[j]) < make_pair(D[i][s], N[s])) s
= j;
                pivot(i, s);
            }
```

```
        }
        bool ok = simplex(1);
        x.assign(n, 0);
        for (int i = 0; i < m; i++) if (B[i] < n) x[B[i]] = D[i][n + 1];
        return ok ? D[m][n + 1] : numeric_limits<T>::infinity();
    }
};

int main() {
    int n, m;
    cin >> n >> m;
    vector<double> c(n), b(m);
    vector<vector<double>> A(m, vector<double>(n));
    for (auto& x : c) cin >> x;
    for (int i = 0; i < m; i++) {
        for (auto& x : A[i]) cin >> x;
        cin >> b[i];
    }
    LPSolver lp(A, b, c);
    vector<double> x;
    double val = lp.solve(x);
    if (val == -numeric_limits<double>::infinity()) cout << "INFEASIBLE\n";
    else if (val == numeric_limits<double>::infinity()) cout << "UNBOUNDED\n";
    else {
        cout << fixed << setprecision(9) << val << "\n";
        for (double xi : x) cout << xi << " ";
        cout << "\n";
    }
}
```

7.4 Maximum flow with lower bounds

Input: `n m s t` (1-indexed vertices), then m edges `u v lo hi`. Output: the maximum s–t flow satisfying all bounds, or `infeasible`.

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct Dinic {
    struct E { int to; ll cap; };
    int n; vector<E> e; vector<vector<int>> g; vector<int> level, it;
    Dinic(int n) : n(n), g(n), level(n), it(n) {}
    int addEdge(int u, int v, ll c) {
        g[u].push_back(e.size()); e.push_back({v, c});
        g[v].push_back(e.size()); e.push_back({u, 0});
        return e.size() - 2;
    }
    bool bfs(int s, int t) {
        fill(level.begin(), level.end(), -1); level[s] = 0;
        queue<int> q; q.push(s);
        while (!q.empty()) {
```

```
            int u = q.front(); q.pop();
            for (int id : g[u]) if (e[id].cap > 0 && level[e[id].to] < 0)
                level[e[id].to] = level[u] + 1, q.push(e[id].to);
        }
        return level[t] >= 0;
    }
    ll dfs(int u, int t, ll f) {
        if (u == t) return f;
        for (int& i = it[u]; i < (int)g[u].size(); i++) {
            int id = g[u][i], v = e[id].to;
            if (e[id].cap > 0 && level[v] == level[u] + 1) {
                ll got = dfs(v, t, min(f, e[id].cap));
                if (got) { e[id].cap -= got; e[id ^ 1].cap += got; return got; }
            }
        }
        return 0;
    }
    ll maxflow(int s, int t) {
        ll flow = 0;
        while (bfs(s, t)) {
            fill(it.begin(), it.end(), 0);
            while (ll f = dfs(s, t, LLONG_MAX)) flow += f;
        }
        return flow;
    }
};

// Edge u->v must carry flow in [lo, hi].
struct BoundedFlow {
    int n; Dinic d; vector<ll> excess; vector<int> ids; vector<ll> lows;
    BoundedFlow(int n) : n(n), d(n + 2), excess(n) {}
    void addEdge(int u, int v, ll lo, ll hi) {
        ids.push_back(d.addEdge(u, v, hi - lo)); lows.push_back(lo);
        excess[v] += lo; excess[u] -= lo;
    }
    bool circulation() {                        // feasible circulation?
        int S = n, T = n + 1; ll need = 0;
        for (int v = 0; v < n; v++) {
            if (excess[v] > 0) d.addEdge(S, v, excess[v]), need += excess[v];
            if (excess[v] < 0) d.addEdge(v, T, -excess[v]);
        }
        return d.maxflow(S, T) == need;
    }
    ll maxFlow(int s, int t) {                  // max s-t flow respecting bounds; -1 if
infeasible
        int back = d.addEdge(t, s, LLONG_MAX / 4);
        if (!circulation()) return -1;
        ll base = d.e[back ^ 1].cap;            // flow currently on t->s
        d.e[back].cap = d.e[back ^ 1].cap = 0;  // delete the t->s edge
        return base + d.maxflow(s, t);
    }
    ll flowOn(int i) const { return lows[i] + d.e[ids[i] ^ 1].cap; }
};

int main() {
```

```
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    BoundedFlow bf(n);
    for (int i = 0; i < m; i++) {
        int u, v; ll lo, hi;
        cin >> u >> v >> lo >> hi;
        bf.addEdge(u - 1, v - 1, lo, hi);
    }
    ll ans = bf.maxFlow(s - 1, t - 1);
    if (ans < 0) cout << "infeasible\n";
    else cout << ans << "\n";
}
```

8. Common mistakes

- **Hungarian with `int`.** Potentials and the running dual sum can exceed the cost range. Use `long long`.
- **Hungarian with n > m.** The loop never terminates or reads garbage. Transpose first.
- **Huge "forbidden" costs.** `a - u - v` overflows if `BIG` is near `LLONG_MAX`. Use something like `1e15` and check the result.
- **Blossom state.** `base`, `par` and `used` must be reset for every search. Forgetting the `match[v] == to` skip makes the search walk back along the matched edge.
- **Blossom with self-loops or duplicate edges.** Self-loops must be dropped (the main above does). Duplicates are harmless.
- **Simplex on badly scaled data.** Mixing 10^{9} and 10^{-3} in one tableau leads to wrong pivots. Rescale rows and columns.
- **Rounding an LP answer to integers** when the matrix is not totally unimodular. The fractional optimum can be strictly better than any integral solution.
- **Lower bounds.** Forgetting the `t → s` edge for s–t problems, checking the T′ side instead of the S′ side, forgetting to add `l_i` back when reporting flows, or running the final `max-flow(s, t)` while the `t → s` edge is still present (which gives an infinite flow).

9. Practice problems

1. CSES 2129 — *Task Assignment* (Hungarian): https://cses.fi/problemset/task/2129
2. Library Checker — *Assignment Problem*: https://judge.yosupo.jp/problem/assignment
3. Library Checker — *Matching on General Graph*: https://judge.yosupo.jp/problem/general_matching
4. Library Checker — *Matching on General Weighted Graph* (notebook-only implementation): https://judge.yosupo.jp/problem/general_weighted_matching
5. UOJ #79 — general graph maximum matching: https://uoj.ac/problem/79
6. Codeforces 375E — *Red and Black Tree* (LP via simplex): https://codeforces.com/problemset/problem/375/E

7. Codeforces 704D — *Captain America* (flow with lower bounds): https://codeforces.com/problemset/problem/704/D
8. LOJ #115 — feasible circulation with bounds: https://loj.ac/p/115
9. LOJ #116 — maximum flow with bounds: https://loj.ac/p/116
10. LOJ #117 — minimum flow with bounds: https://loj.ac/p/117
11. ICPC WF 2015 — *Catering* (min-cost flow), on the Kattis archive.

10. Further reading

- cp-algorithms, "Hungarian algorithm for solving the assignment problem": https://cp-algorithms.com/graph/hungarian-algorithm.html
- cp-algorithms, "Flows with demands": https://cp-algorithms.com/graph/flow_with_demands.html
- KACTL `Simplex.h`, `GeneralMatching.h` (Tutte matrix) and `WeightedMatching.h`: https://github.com/kth-competitive-programming/kactl
- A. Schrijver, *Combinatorial Optimization: Polyhedra and Efficiency* — the definitive reference for matching polytopes and total unimodularity.
- V. Chvátal, *Linear Programming* — a readable treatment of simplex, degeneracy and duality.
- J. Edmonds, "Paths, trees, and flowers", Canadian Journal of Mathematics, 1965 — the original blossom paper.